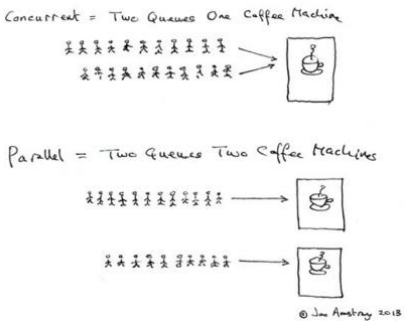
Programmazione Concorrente e Algoritmi Distribuiti

Introduzione

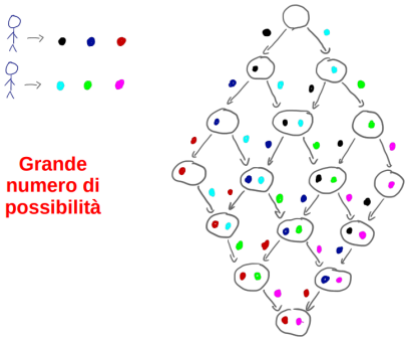


Perdendo come esempio una macchina del caffè, un programma concorrente può essere rappresentato da due code ed una sola macchina del caffè, mentre un programma parallelo può essere rappresentato da due macchine del caffè e due rispettive code.

Motivi per usare sistemi concorrenti/distribuiti:

- Efficienza
- Per distribuire dati
- Per condividere risorse
- Per usare reti di computer
- Per comunicare

La Legge di Moore "il numero di transistor nei processori raddoppia ogni 18 mesi" non è più applicabile per problemi di limiti fisici, quindi, visto che un processore non basta più siamo passati ad architetture multiprocessore.



È un processo difficile concettualmente in quanto più processi abbiamo, più è grande il numero di possibilità dell'ordine di esecuzione. Prendendo come esempio n uomini, che devono estrarre da un cappello m palline ciascuno, quante possibilità (ossia quanti ordini possibili) ci sono? La formula è $\frac{(n \cdot m)!}{(m!)^n}$. Questo vuol dire che più palline (processi) ci sono, e più le possibilità cresceranno.

Un programma concorrente è un insieme di programmi sequenziali (processi) che si eseguono in parallelo e che comunicano.

- Parallelismo: programmi che si eseguono su processori diversi, ma anche programmi multi-task che si eseguono sullo stesso processore (illusione di parallelismo)
- Comunicazione: scambio di dati e sincronizzazione che avviene tramite memoria condivisa o via scambio di messaggi.

Esecuzione di un programma concorrente:

- Insieme di processi (programmi sequenziali)
- Ciascun processo dispone di un insieme di istruzioni atomiche
- Ogni processo ha un 'puntatore' verso la prossima istruzione da eseguire
- L'esecuzione del programma concorrente è un arbitrario intreccio (interleaving) delle istruzioni dei vari processi
- Durante un'esecuzione, la prossima istruzione è scelta tra quelle puntate dai vari processi

Questo vuol dire che ci sono più scenari possibili.

I processi hanno un accesso comune alla memoria, che avviene tramite variabili semplici condivise che possono essere lette o scritte. Bisogna fare attenzione a come viene gestito l'accesso a queste variabili (transazione atomica o non atomica). I vari casi sono:

- Variabili in lettura e scrittura per più processi, 'Multiple Reader/Multiple Writer'
- Variabili in lettura per più processi ma in scrittura per un solo processo, 'Multiple Reader/Single Writer'

Processi e Thread

Un processo è un programma (statico) in corso di esecuzione (dinamico). La sua esecuzione necessita un ambiente, con uno spazio di indirizzamento dedicato e un metodo di input e output. Più processi possono eseguirsi sulla stessa macchina in un modo quasi simultaneo SE il sistema rispetta il tempo condiviso o è multi-task. Ciò è valido anche con un solo processore; il sistema operativo è responsabile dell'allocazione delle risorse (memoria, tempo processore, input/output, ecc). Un processo non ha accesso allo spazio di indirizzamento degli altri processi. Abbiamo l'illusione del parallelismo, ossia che ogni processo pensa di avere tutte le risorse disponibili (che vengono invece controllate dal sistema operativo).

In C un processo vie3ne creato grazie alla funzione fork: **pid_t fork(void);**
Questa funzione crea un processo figlio e restituisce un identificatore del processo PID. FOrk restituisce 0 per il figlio e restituisce il PID del figlio al padre.

- **pid_t getpid(void);** per avere il PID del processo corrente
- **pid_t getppid(void);** per avere il PID del processo padre del processo corrente

Quindi per capire se ci si trova nel processo figlio si fa **if(fork( )==0) {...}**
È importante ricordare che le variabili non sono condivise tra padre e figlio e che, alla creazione del figlio, lo spazio di indirizzamento del padre viene copiato.

Per far eseguire un programma (di solito al figlio) si può usare la funzione execvp:

int execvp(const char *file, char *const argv[]);

L'array argv contiene il nome della command nel primo campo, gli argomenti nei campi seguenti e deve finire con NULL. Execvp non crea nessun nuovo processo: semplicemente lo spazio di indirizzamento viene rimpiazzato con quello del nuovo programma.

Come si comunica tra processi?

Per comunicare tra processi, si possono usare dei tubi (pipe), dei canali unidirezionali considerati come dei descrittori di file. Un processo può scrivere ad un'estremità e un altro può leggere dall'altra. Poiché sono considerati come descrittori di file, quando un processo crea un figlio, quest'ultimo recupera la tabella con i descrittori del processo padre e ha accesso al tubo.



La funzione **int pipe(int fds[2])** crea un tubo e mette le sue due estremità nell'array fds: si scrive su fds[1] e si legge su fds[0]. Dopo il fork, entrambi processi possono scrivere e leggere, ma affinché funzioni correttamente, è necessario chiudere il "lato" non utilizzato.
Ad esempio, se il processo padre scrive, deve chiudere fds[0] per primo, e se il processo figlio legge, deve chiudere fds[1]. È necessario chiudere bene appena si è finito di usare il tubo.

Come si condividono delle variabili?

Per farlo si usano dei processi leggeri: i thread. Un thread è un filo d'esecuzione all'interno di un programma. Il thread è eseguito da un processo, che può gestire più thread, ed in quel caso si parla di processo multi-thread.

Ogni thread è diverso e presenta i seguenti attribuiti: un puntatore di esecuzione o PC (Program Counter) ed uno stack d'esecuzione.

In C, esiste una libreria 'classica' per la creazione e la manipolazione di thread: la libreria POSIX. Per usarla, bisogna includerla nel codice: **#include <pthread.h>**

Per creare uno thread, si usa la funzione:

```
int pthread_create(
        pthread_t *thread, //Per ricordare i dati del nuovo thread
        const pthread_attr_t *attr, // Attributi del thread
        void *(*start_routine) (void *), // Funzione da eseguire
        void *arg); // Argomenti della funzione
```

In pratica, per gli attributi, si può mettere NULL per utilizzare gli attributi predefiniti. Questi attributi permettono, per esempio, di definire una politica di ordinamento per lo scheduler. Il codice eseguito dallo thread è dato dalla funzione start_routine. Appena la funzione pthread_create termina, il thread è creato e può iniziare l'esecuzione.

Per passare degli argomenti alla funzione al momento della creazione dello thread, si inseriscono come ultimo argomento della funzione pthread_create.

A volte è necessario attendere la fine dell'esecuzione degli thread creati; se il programma principale termina prima, allora i thread vengono distrutti. Per questo scopo si può usare la funzione **int pthread_join(pthread_t thread, void **value_ptr)**: il primo argomento corrisponde a quello creato da pthread_create, mentre value_ptr è il valore ritornato dallo thread chiamando la funzione **void pthread_exit(void *value_ptr), che** provoca la terminazione del thread.

Per condividere una variabile, si può usare una variabile globale o passare il loro indirizzo come argomento di pthread_create. Le variabili devono essere dichiarate come 'volatile' (esempio: volatile int a=0;).

Si deve prestare attenzione con i dati condivisi. I thread si eseguono in parallelo e non è possibile fare ipotesi sullo scheduler (è lui che interrompe gli thread e che decide quale thread deve essere eseguito in ogni momento).

Se non si presta attenzione alla manipolazione delle variabili condivise, si possono osservare dei comportamenti strani.

Ad esempio:

- un thread che testa il valore di una variabile intera prima di ridurla
- viene interrotto fra il test e l'update
- un altro thread imposta il valore della variabile a 0

- il primo thread imposta la variabile a -1, mentre forse non era quello che si voleva ottenere

Interleaving e Mutua Esclusione

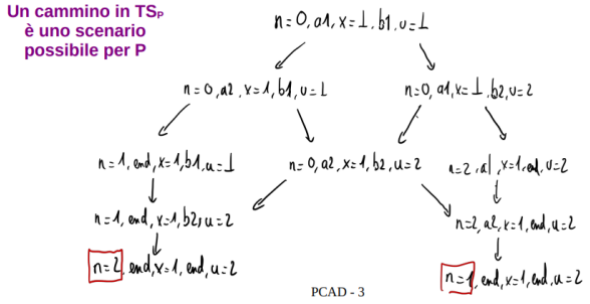
Un'istruzione atomica è un'istruzione che viene eseguita completamente senza essere interrotta. Non è sempre facile definirle. La correzione dei programmi dipende dalle ipotesi di atomicità, che possono cambiare a seconda del compilatore, del processore, ecc.
Una regola: Evitare di manipolare più di una occorrenza di una variabile condivisa fra processi.
Esistono diversi casi:

- registri atomici → operazione di lettura e di scrittura atomiche
- test-and-set → si può testare E modificare una variabile senza essere interrotti
- swap → si può scambiare il valore di un registro locale e di uno condiviso in un modo atomico

Interleaving (semantica di intreccio)
Uno stato di un programma concorrente P è una tupla che include un puntatore alle istruzioni per ciascun processo e un valore per ciascuna delle variabili (locale e condivisa).
Per due stati s1 e s2, si scrive s1 → s2 quando è possibile passare da s1 a s2 utilizzando una delle istruzioni puntate in s1 [Esempio: (n=0, a1, x=1, b1, u=2) → (n=2, end, x=1, end, u=2)]

Il diagramma degli stati di P è il sistema di transizione (ovvero il grafo) $TS_P = (S, \rightarrow, s0)$ dove:

- S è l'insieme degli stati di P
- → è la relazione di transizione inclusa in S x S
- s0 è lo stato iniziale di P



Una semantica ragionevole:

- Per i sistemi con un processore condiviso fra più processi, c'è una successione d'istruzioni di un processo, poi di un altro, ecc
- Per i sistemi con più processori, ogni processo è legato ad un processore; la semantica d'intreccio non corrisponde esattamente alla realtà (ci sono delle azioni parallele), ma è corretta se non ci sono conflitti sulle risorse (evitare l'accesso simultaneo ai dati condivisi)
- Per i sistemi distribuiti (diversi computer) non ci sono variabili condivise, ma dei canali di comunicazione; è Molto diversa dai primi due casi, ma la semantica rimane interessante se si aggiungono l'invio e la ricezione di messaggi.

Il problema centrale è il problema della mutua esclusione. La mutua esclusione permette di garantire che, in un determinato momento, un processo ha un accesso esclusivo a delle risorse comuni.

Il problema: N ≥ 2 processi eseguono in modo continuo due sequenze d'istruzioni:

- Una sezione non critica (SNC)
- Una sezione critica (SC)

Per fare in modo che la mutua esclusione sia garantita in SC, ad ogni istante c'è al massimo un processo in sezione critica.
Per risolvere questo problema, useremo delle variabili condivise (che non verranno usate in SC e SNC), all'interno:

- di un pre-protocollo prima di accedere a SC
- di un post-protocollo dopo l'accesso a SC

Proprietà da verificare:

- Mutua esclusione: al massimo un processo in SC ad ogni momento
- Assenza di deadlock: se più processi provano di accedere alla SC nello stesso momento, non devono essere bloccati nel pre-protocollo e al meno un processo deve riuscirci
- Assenza di starvation: se un processo desidera accedere alla sua SC, prima o poi ci riuscirà
- Attesa limitata: se un processo deve attendere per accedere alla sua SC, è possibile limitare il numero di volte in cui dovrà concedere il posto a un altro processo tramite una costante

Attesa limitata → Assenza di starvation → Assenza di deadlock

Mutex e Variabile Condizionale

Per proteggere i dati, l'accesso ai dati condivisi deve essere protetto. In C, si possono usare dei lock (mutex). Il principio di funzionamento è:

- Uno thread che desidera accedere a un dato condiviso richiede il lock
- Se il lock è libero, il thread prosegue
- Se il lock è preso, il thread rimane bloccato finché il lock sia liberato
- Quando ha finito di accedere ai dati condivisi, libera il lock

Questi lock sono condivisi fra i threads, ed è buona pratica che sia il thread che ha preso il lock che deve liberarlo.
La libreria POSIX ha dei lock di tipo: **pthread_mutex_t lock;**
La prima cosa da fare è di inizializzare il lock: l**ock=PTHREAD_MUTEX_INITIALIZER;** (solo alla dichiarazione) oppure **pthread_mutex_init(&lock,NULL)**.

- Per prendere il lock: **int pthread_mutex_lock(pthread_mutex_t *mutex);**
- Per liberarlo: **int pthread_mutex_unlock(pthread_mutex_t *mutex);**

Queste due funzioni rstituiscono 0 in caso di esito positivo
Per distruggere il lock: **pthread_mutex_destroy(&lock).**
I locks permettono di proteggere l'accesso a dei dati condivisi e inducono una certa forma di sincronizzazione: per accedere alla sezione critica, se un altro thread è già dentro, devo aspettare che esca e rimango bloccato in attesa. Più thread possono essere in attesa su un lock di un mutex:

- uno dei thread in attesa, prenderà il lock
- gli altri rimarranno bloccati in attesa
- in che ordine? non è specificato dalla norma POSIX; quindi, si deve assumere che possa avvenire in qualsiasi ordine

Importante: deve essere il thread che possiede il lock a fare unlock!!

A volte è necessario avere una forma di sincronizzazione più forte.
Un thread deve aspettare che un altro thread abbia finito di fare qualcosa. Ad esempio, il thread 1 produce un valore che deve essere consumato dal thread 2: questo è un caso di un Producer/Consumer problem.
Una soluzione: ogni volta che il producer produce un nuovo valore, imposta un boolean condiviso a true, mentre il consumer aspetta che il boolean sia impostato a true per consumare.
Ma cosa significa aspettare? Può fare un ciclo while finché il boolean è impostato su true: ogni volta prende il lock, verifica il boolean e rilascia il lock. Questo procedimento si chiama active waiting (attesa attiva), che non è ottimale in termini di concorrenza, perché utilizza risorse senza fare nulla.

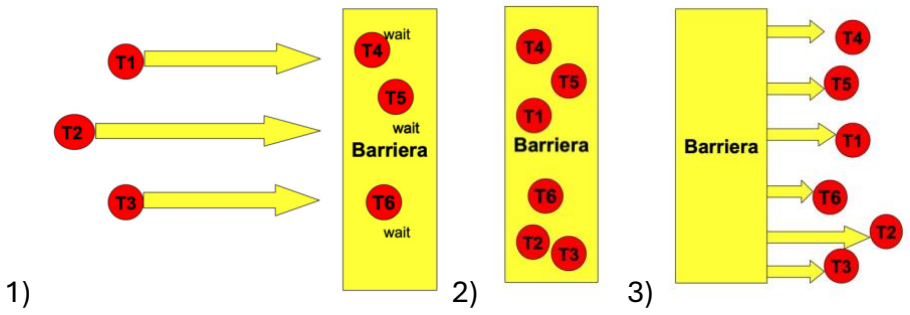
Come evitare l'attesa attiva?
Per evitare l'attesa attiva si possono usare le variabili condizionali (monitor del C). Una variabile condizionale è associata ad un mutex, permette a un thread di aspettare su questa variabile (in modo passivo) e permette a un thread di svegliare altri thread che stanno aspettando su questa variabile. Spesso le variabili condizionali sono associate a delle condizioni su variabili condivise da sorvegliare: se una condizione necessaria allo thread non è verificata, il thread aspetta in un modo passivo finché la condizione viene verificata.

- Dichiarazione: **pthread_cond_t varcond;**
- Inizializzazione: **pthread_cond_init(&varcond,NULL);**
- Per aspettare: **pthread_cond_wait(&varcond,&lock);** // Lock è un lock preso al momento dello wait; questa funzione libera lock e 'addormenta' il thread.
- Per svegliare:
    - **pthread_cond_signal(&varcond);** // 'sveglia' al meno un thread addormentato sur varcond (se ce ne sono, altrimenti non fa nulla)
    - **pthread_cond_broadcast(&varcond);** // 'sveglia' tutti gli thread addormentati sur varcond (se ce ne sono, altrimenti non fa nulla)

Quando uno thread viene risvegliato, viene inserito in una lista d'attesa per essere scelto dallo scheduler e quando sarà il suo torno la prima cosa che farà sarà prendere il lock (ed attedere se il lock è già stato preso).

Barriere
Le barriere sono un meccanismo di sincronizzazione utilizzato per esempio nel calcolo parallelo.
Idea: una barriera rappresenta un punto di sincronizzazione per N thread

- I thread che arrivano alla barriera aspettano gli altri
- Solo quando tutti gli N thread arrivano alla barriera allora possono proseguire



- Dichiarazione: **pthread_barrier_t bar;**
- Inizializzazione: **pthread_barrier_init(&bar, NULL, 5);** //per dire 5 threads devono arrivare
- Per aspettare: **pthread_barrier_wait(&bar);** // se non ci sono 5 threads in attesa, blocca; quando il quinto thread arriva, tutti vengono svegliati

Possono essere implementate facilmente con variabile condizionale.

Problema della sezione critica

Ci sono diversi algoritmi per risolvere il problema della sezione critica.

Algoritmo di Dekker:

```
int turn=1; //variabili condivise
bool D1=false;
bool D2=false;
```

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: while(D2==true){
p4:   if(turn==2){
p5:     D1=false;
p6:     while(turn!=1){}
p7:     D1=true;}}
p8: SC
p9:  turn=2;
p10:D1=false;}
```

```
Process P2
while(true){
q1: SNC
q2: D2=true;
q3: while(D1==true){
q4:   if(turn==1){
q5:     D2=false;
q6:     while(turn!=2){}
q7:     D2=true;}}
q8: SC
q9:  turn=1;
q10:D2=false;}
```

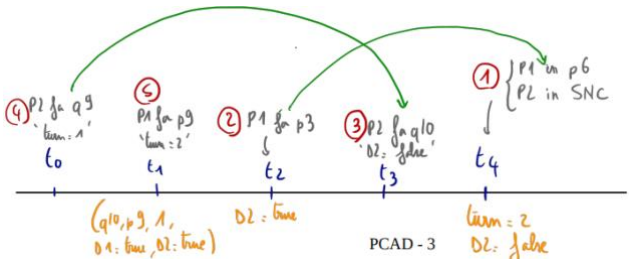
Un invariante è una proprietà che è vera in tutti gli stati. Un invariante induttivo è una proprietà che è vera in uno stato e per tutti gli stati (anche quelli non raggiungibili): se è vera allora è vera in ogni stato successivo.

- L'algoritmo di Dekker verifica gli invarianti (induttivi) successivi:
  - turn==1 or turn==2
  - (p3 or p4 or p5 or p8 or p9 or p10) <=> D1==true
  - (q3 or q4 or q5 or q8 or q9 or q10) <=> D2==true
- L'algoritmo di Dekker verifica la mutua esclusione. Supponiamo che P1 arriva in SC (quindi in p8): questo vuol dire che allora D2==false (usciamo del while). Grazie all'invariante **(q3 or q4 or q5 or q8 or q9) <=> D2==true,** sappiamo che P2 non può essere anche lui in q8.
- L'algoritmo di Dekker verifica l'assenza di deadlock. Supponiamo che c'è un deadlock, quindi P1 e P2 sono entrambi 'bloccati' nel pre-protocollo. Supponiamo che turn==1, allora P1 finirà per essere in loop fra p3 e p4, mentre P2 finirà per essere in loop su q6 (perché turn==1). Quindi finiremmo per avere D2=false grazie all'invariante (q3 or q4 or q5 or q8 or q9) <=> D2==true. Ma allora P1 non può essere in loop fra p3 e p4, quindi c'è assenza di deadlock.

- L'algoritmo di Dekker verifica l'assenza di starvation sotto due ipotesi: l'equità tra processi (ossia che se un processo può eseguire un'istruzione, la eseguirà prima o poi); la SC termina sempre e finisce con l'esecuzione del post-protocollo.
  Supponiamo che P1 sia in starvation, dunque P1 è quindi bloccato nel suo pre-protocollo. Dove si trova P2? Non può essere bloccato nel pre-protocollo (per l'assenza di deadlock). Se non è bloccato nella sua SNC e richiede la SC, allora dopo aver avuto accesso alla SC:
  - mette turn a 1 dando la priorità a P1
  - turn non cambierà più, quindi P1 non può essere bloccato in p6
  - allora P1 è bloccato fra p3 e p4 e P2 finirà bloccato in q6, visto che turn==1 e D1==true. Ma allora abbiamo un deadlock: contradizione!

  Se P2 è bloccato nella sua SNC:
  - t4: tempo in cui P2 è bloccato in SNC e P1 è bloccato nel pre-protocollo: D2==false per tutto il futuro, quindi P1 è bloccato in p6 e turn==2
  - t2: ultimo momento in cui P1 ha fatto p3 (t2<t4): come P1 è in p6 in t4, in t2 abbiamo D2==true

  Tra t2 e t4 D2 è cambiato, quindi in t3 P2 ha fatto q10 (t2<t3<t4). Prima di t3, al tempo t0, P2 ha fatto q9, mettendo turn=1 (t0<t3, P2 non fa nulla tra t0 e t3). Dopo t0, in t1, P1 ha fatto p9 (t1<t2). In t1 abbiamo lo stato (p9, p10, turn=1, D1==true, D2==true), che però non è raggiungibile.



<u>Algoritmo di Peterson:</u>

L'algoritmo di Peterson è più facile da adattare per N>2 processi ed è più facile dimostrare la sua correttezza.

```
int turn=1; //variabili condivise
bool D1=false;
bool D2=false;
```

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: turn=2;
p4: while(turn!=1 &&D2==true){}
p5:  SC
p6:D1=false;}
```

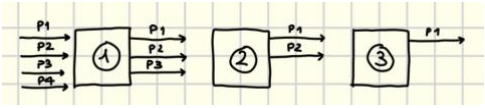
```
Process P2
while(true){
q1: SNC
q2: D2=true;
q3: turn=1;
q4: while(turn!=2 &&D1==true){}
q5:  SC
q6:D2=false;}
```

- L'algoritmo di Peterson verifica la mutua esclusione. Supponiamo che P1 arrivi in SC dopo P2. Allora D2=D1=true (P1 e P2 sono in p4 e p5) e turn=1 (perché P1 è uscito dal while). Come ha fatto P2 a passare il while?
  - con D1=false? Nel frattempo P1 ha impostato D1=true e turn=2, ma allora P1 non può passare il while
  - con turn=2? Allora turn rimane sempre uguale a 2 finché P2 è in SC e P1 non può passare il while

- L'algoritmo di Peterson verifica l'assenza di deadlock. Se c'è deadlock, P1 è bloccato in p4 e P2 in q4. L'unico modo sarebbe avere **(turn!=1 && D2==true) && (turn!=2 && D1==true)**, ma è impossibile perché turn prende come valore o 1 o 2!
- L'algoritmo di Peterson verifica l'assenza di starvation sotto due ipotesi: l'equità tra processi (ossia che se un processo può eseguire un'istruzione, la eseguirà prima o poi); la SC termina sempre e finisce con l'esecuzione del post-protocollo. Supponiamo starvation di P1, allora: D1=true per sempre e D2=true e turn=2. Visto che D2=true, vuol dire che P2 è in q3, q4, q5, q6, quindi vuol dire che in qualche modo il turno diventerà di 1 prima o poi.

<u>Passare da 2 a N>2 processi</u>
Ci sono diversi algoritmi per passare da 2 a N processi:

Filter Lock: ci sono N-1 sale d'attesa che bloccano almeno un processo, e bisogna ad attraversare queste sale (detti level).



Per ogni level:
- Tra i processi che provano a passare, almeno uno riesce
- Se ci sono più processi che provano, almeno uno viene bloccato

Abbiamo:
- level[id] = j → processo P[id] prova a passare la 'sala d'attesa' j
- victim[j] = id → il processo che verrà bloccato nella sala j è id (se piu processi provamo a passare)

Quindi, un processo può andare avanti se non è la vittima o se non c'è nessuno ad un livello superiore → while(∃ k ≠ id.level[k] ≥ j && victim[j] == id) {}

Abbiamo $num_{\geq j} \leq$ n-j per ogni 0 ≤ j ≤ n-1.

L'algoritmo di Filter Lock verifica la mutua esclusione. L'algoritmo di Filter Lock verifica l'assenza di starvation sotto due ipotesi: l'equità tra processi (ossia che se un processo può eseguire un'istruzione, la eseguirà prima o poi); la SC termina sempre e finisce con l'esecuzione del post-protocollo. L'assenza di starvation implica l'assenza di deadlock, MA l'assenza di deadlock non implica l'assenza di starvation!

Ogni processo al livello j ≥ 1 arriverà al livello j+1 o in SC se j=N-1.

```
Process P[id]
while(true){
p1: SNC
p2:for (int j=1; j<N; j++){
p3:  level[id]=j
p4:  victim[j]=id
p5:  while( ∃k≠id.level[k]≥ j && victim[j]==id){}
    }
p6:SC
p7:level[id]=0;}
```

Il Torneo: ad ogni turno, il numero massimo di processi che va avanti è diviso per 2. Abbiamo n=2^k processi, e facciamo un torneo con eliminazione in $\log(n) = k$ giri. Dopo il primo giro, sono eliminati n/2 processi, dopo il secondo giro sono eliminati n/4 processi, ecc. I giri sono

numerati da 0 a log(n)-1. Ogni competizione tra due processi è gestita da un algoritmo di mutua esclusione per due processi.

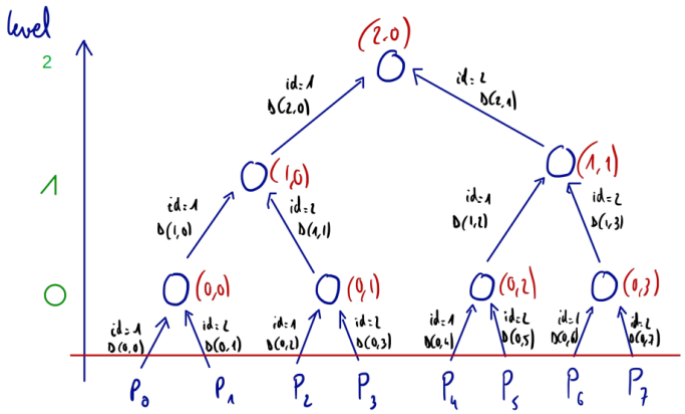
Ogni 'partita' è identificata da un giro/livello le e da un numero k con: **le ϵ {0, ..., log(n)-1} e k ϵ {0, ..., 2 log(n)-1-le-1}.**

Per usare l'algoritmo di Peterson, dobbiamo avere l'equivalente delle variabili condivise:

- int turn(le,k : chi è prioritaria per la partita
- bool D(le,2k): il processo di sinistra che vuole accedere alla sezione criticata
- bool D(le,2k+1) il processo di destra che vuole accedere alla sezione criticata

Ogni processo deve anche sapere se è il processo di sinistra o di destra nella partita che sta giocando. Per ogni processo abbiamo quindi:

- le: il giro corrente
- k: il numero della partita nel giro l
- id ϵ {1,2} che indica se il processo è a sinistra (1) o a destra (2)



L'algoritmo del torneo verifica la mutua esclusione. Se due processi sono insieme in SC, sono passati entrambi dalla stessa partita in un determinato momento. Sia (le,k) quella partita; poiché i due processi provengono da due rami diversi hanno due id diversi. Tuttavia, se uno ha superato il while e l'altro l'ha raggiunto senza eseguire il post-protocollo, significa che l'algoritmo di Peterson è falso....

L'algoritmo del torneo verifica l'assenza di starvation sotto due ipotesi: l'equità tra processi (ossia che se un processo può eseguire un'istruzione, la eseguirà prima o poi); la SC termina sempre e finisce con l'esecuzione del post-protocollo. Se un processo è bloccato alla partita (le,k), a un certo punto diventerà prioritario, una volta che l'altro processo partecipante alla partita avrà eseguito il suo post-protocollo.

```
Process P[i]
while(true){
p1: SNC
p2: k=i
p3: for(int le=0; le<log(n); le++){
p4:    id=k%2+1 //id vale 1 o 2
p5:    k=k/2
p6:    D(le,2k+(id-1))=true
p7:     turn(le,k)=3-id
p8:     while(turn(le,k)!=id &&D(le,2k+2-id)==true){}
    }
p9: SC
p10: for(int le=log(n)-1; le>=0; le--){
p11:     D(le,i/2^le)=false}
```

Algoritmo del fornaio (bakery algorithm): algoritmo di mutua esclusione per N≥2 processi. Usa delle variabili proprie (multiple readers/single writer): ogni processo ha le sue variabili che possono essere lette ma non modificate dagli altri processi. È l'algoritmo usato nell'amministrazione, o dalle banche: un nuovo 'cliente' che vuole avere accesso alle risorse prende un numero superiore ai numeri dei clienti già in attesa, e il cliente prioritario è quello che ha il numero più piccolo.

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:    if(j!=id){
p7:       while(choosing[j]){}
p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id))){}}}
p9:   SC
p10:nb[id]=0
```

L'algoritmo del fornaio verifica la mutua esclusione. Scegliamo un processo P[i] che arriva in SC ad un istante t2. Dobbiamo dimostrare che a questo istante, nessun altro processo P[k] (con k≠i) è in SC. Scegliamo un k≠i e vediamo dove è P[k] in t2. Usiamo altri due istanti:

- t0: P[i] passa il **while(choosing[k]){}**
- t1: P[i] passa il **while(nb[k]!=0 && (nb[k],k) << (nb[i],i))){}**

Dove può essere P[k] a t0?

- in p3, p4? no perché a t0, P[i] passa **while(choosing[k]){}** quindi **choosing[k]=false**
- in p1, p2? quindi quando P[k] eseguirà il suo pre-protocollo, non potrà essere in SC prima che P[i] ne esca
- in p5, p6, p7, p8, p9, p10? ma allora nb[k] è fisso e abbiamo due casi
    - **(nb[k], k) << (nb[i], i)**: allora nb[k] deve essere messo a 0 prima di t1 e P[k] passerà quindi in p1, p2 (come prima)
    - **nb[i], i) << (nb[k], k):** il calcolo di nb[i] inizia prima della fine di quello di nb[k]; con **while(choosing[i]){}**, P[k] dovrà aspettare che nb[i] sia calcolato e verrà bloccato in p8.
- P[k] non può essere in SC in t2

L'algoritmo del fornaio verifica l'assenza di deadlock. Se tanti processi si bloccano, si ritrovano tutti bloccati in p8. Ma dato che l'ordine << è totale (ogni processo ha un numero diverso), c'è per forza un i tale quale (nb[i], i) è l'elemento più piccolo e lui passerà: quindi non ci sarà un deadlock.

L'algoritmo del fornaio verifica l'assenza di starvation sotto le ipotesi di equità fra i processi (se un processo può eseguire una istruzione, la eseguirà un giorno, e la SC termina sempre e finisce con l'esecuzione del post-protocollo. Supponiamo che P[i] rimanga bloccato nel suo pre-protocollo: è bloccato per forza in p8 per un j, nb[j] > 0 && (nb[j], j) << (nb[i], i). P[j] finirà in SC (non c'è deadlock) e dopo P[j] rimarrà in SC, e nb[j]=0. Poi P[j] rifarà il suo pre-protocollo e il calcolo di nb[j] prenderà in considerazione il valore di nb[i] e P[i] passerà.

Nel l'algoritmo del fornaio i valori nel nb possono crescere all'infinito (anche con solo due processi)!

Test-and-set: Un test-and-set è una variabile boolean con due operazioni atomiche:

- test-and-set: legge il valore della variabile, la mette a true, e ritorna il valore letto
- reset: mette false nella variabile

Questo algoritmo verifica mutua esclusione e l'assenza di deadlock, ma non quella di starvation.

```
Process P
while(true){
p1: SNC
p2: while(x.test-and-set()==true){}
p5:  SC
p6: x.reset()}
```

Semafori

I semafori sono oggetti di alto livello per modellare delle risorse, introdotti da Dijkstra nel 1965. Abbiamo un numero conosciuto di risorse; i processi che ne hanno bisogno possono chiederle, e se non ce ne sono i richiedenti vengono messi in attesa. Ci sono diversi modi per far aspettare i processi, secondo il tipo di semafori. Un semaforo può essere intero o booleano.

I semafori possono essere distinti in tre tipi:

- I semafori deboli (weak): i processi in attesa sono bloccati, e quando la risorsa torna disponibile SOLO UN processo in attesa viene svegliato
- I semafori forti (strong): i processi in attesa sono bloccati ed inseriti in una struttura FIFO (First In First Out), e quando la risorsa torna disponibile, il processo in attesa da più tempo viene svegliato
- I semafori busy-wait: i processi rimangono attivi e testano di continuo se la risorsa è disponibile

Ogni semaforo S è composto da due campi:

- S.V: un intero (o booleano) che rappresenta il valore di S
- S.L: la lista dei processi in attesa

Per i semafori forti, S.L è una struttura FIFO fair, per la quale si può assumere un limite sul numero di volte in cui un processo in attesa può essere 'superato' da altri. Per i semafori deboli non ci sono ipotesi di fairness su S.L, sappiamo solo che sarà sempre un processo bloccato a ricevere la risorsa.

Ci sono due operazioni atomiche su un semaforo S:

- S.wait( ), per chiedere una risorsa
- S.signal( ), per rilasciare una risorsa

```
S.wait(){
if (S.V>0){S.V--}
else{
  S.L=S.L U{P}
  P.stato=bloccato
}}
```

```
S.signal(){
if (S.L=Ø){S.V++}
else{
  P'=S.L.estrarre()
  P'.stato=pronto
}}
```

Gli stati possibili di un processo sono inattivo, pronto, attivo, finito e bloccato. Per i semafori binari, il valore V è bloccato ad 1, quindi fare V++ se vale 1 non fa nulla.

Per i semafori busy-wait:

```
S.wait(){
  while(S.V==0){}
  S.V--
}
```

```
S.signal(){
  S.V++
}
```

Le operazioni wait e signal sono atomiche, ma in questo caso dobbiamo assumere che durante il while, il processo può essere interotto.

Proprietà dei semafori:
Sia k il valore iniziale di S.V all'inizializzazione. Abbiamo sempre (se si usa solo wait e signal per accedere al semaforo):

- $S.V \geq 0$
- $S.V = k + \#S.signal( ) - S.wait( )$

#S.signal() è il numero di chiamate a signal() fatte fino ad ora e #S.wait() è il numero di chiamate a wait() fatte e finite fino ad ora.

Problema della sezione critica

- Con i semafori

```
Process P1
while(true){
p1: SNC
p2: S.wait();
p3:  SC
p4: S.signal();}
```

```
Process P2
while(true){
q1: SNC
q2: S.wait();
q3:  SC
q4: S.signal();}
```

  S: semaforo binario con S.V = 1. Le proprietà di mutua esclusione, assenza di deadlock, assenza di starvation e attesa limitata sono tutte vere, sia che il semaforo sia debole che sia forte.

- Con i semafori e N processi

```
Process P[1]/P[2]/.../P[N]
while(true){
p1: SNC
p2: S.wait();
p3:  SC
p4: S.signal();}
```

  S: semaforo binario con S.V = 1. La mutua esclusione e l'assenza di deadlock sono verificate a priori, ma l'assenza di starvation e l'attesa limitata dipendono se il semaforo è debole o forte: sono verificate se forte, e non verificate se debole.

Esempi di uso dei semafori
Problema di precedenza: una task t deve essere realizzata prima di T'

- S: semaforo binario inizializzato a 0
- T finisce facendo S.signal( )
- T' inizia facendo S.wait( )

Il problema di produttori/consumatori: Abbiamo una comunicazione asincrona: le risorse sono prodotte e messe in un buffer (infinito o finito), e sono consumate da altri processi (se disponibile).

- Buffer infinito

F: struttura FIFO =Ø // variabile condivise
NonVuota : semaforo con S.V=0

```
Process Produttore:
while(true){
p1: d=product();
p2: F.add(d);
p3: NonVuota.signal();}
```

```
Process Consumatore
while(true){
q1: NonVuota.wait();
q2: d=F.get();
q3: consume(d);}
```

Invariante : |F|=NonVuota.V

=> Il consumatore non puo prendere una risorsa non disponibile

- Buffer finito di lunghezza N

F: struttura FIFO =Ø // variabile condivise
NonVuota : semaforo con S.V=0 ← Posti occupati
NonPiena : semaforo con S.V=N ← Posti liberi

```
Process Produttore:
while(true){
p1: d=product();
p2: NonPiena.wait();
p3: F.add(d);
p4: NonVuota.signal();}
```

```
Process Consumatore
while(true){
q1: NonVuota.wait();
q2: d=F.get();
q3: NonPiena.signal();
q4: consume(d);}
```

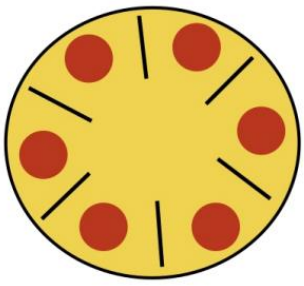
Invariante : N=NonVuota.V + NonPiena.V

Problema dei filosofi:
Abbiamo N filosofi, N bacchette e per mangiare, un filosofo deve avere la sua bacchetta di destra e di sinistra. Dobbiamo fare in modo che siano verificate queste proprietà:

- Un filosofo può mangiare solo con le bacchette alla sua destra e sinistra
- Una bacchetta è presa al massimo da un filosofo
- Assenza di deadlock
- Assenza di starvation

Schema generale :

```
Process Filosofo:
while(true){
p1: pensa;
p2: pre-protocollo
p3: mangia
p4: post-protocollo}
```



Mangia finisce sempre, pensa non necessariamente!

Una soluzione possibile è quella di limitare il numero di filosofi che possono fare il pre-protocollo insieme.

Semaforo bacchette[N]={1,...1}; // variabile condivise
Semaforo ticket=M; //M<N

```
Process Filosofo id: //id ϵ {0,...,N-1}
while(true){
 pensa
 ticket.wait()
 bachette[i].wait()
 bachette[(i+1)%N].wait()
 mangia
 bachette[i].signal()
 bachette[(i+1)%N].signal()
 ticket.signal()
}
```

Vero se ticket è strong o se ticket è weak e M=N-1 →

Proprietà attese

- Un filosofo può mangiare solo con le bacchette alla sua destra e sinistra ✅
- Un bacchetta è presa dal al massimo un filosofo ✅
- Assenza di deadlock ✅
- Assenza di starvation ✅

Semaforo bacchette[N]={1,...1}; // variabile condivise

```
Process Filosofo id: //id ϵ {1,...,N-1}
while(true){
 pensa
 bachette[i].wait()
 bachette[(i+1)%N].wait()
 mangia
 bachette[i].signal()
 bachette[(i+1)%N].signal()
}
```

```
Process Filosofo 0:
while(true){
 pensa
 bachette[1].wait()
 bachette[0].wait()
 mangia
 bachette[1].signal()
 bachette[0].signal()
}
```

Semafori in C:

- Libreria: **include <semaphore.h>;**

- Dichiarazione: **sem_t sem;**
- Inizializzazione: **int sem_init(sem_t *sem, int pshared, unsigned int value);** //pshared to 0 for semaphore shared between thread, value is the initial value
- Operazioni:
    - **int sem_wait(sem_t *sem)**
    - **int sem_post(sem_t *sem)** // signal

Processi e thread in Java

Un processo è un programma (di natura statica) in corso di esecuzione (di natura dinamica). La sua esecuzione necessità un ambiente con uno spazio di indirizzamento dedicato, e input e output (per es. gli standard). Più processi possono essere eseguiti sulla stessa macchina in modo quasi simultaneo se il sistema rispetta il tempo condiviso o è multi-task (vale anche se c'è un solo processore). Il sistema operativo è responsabile dell'allocazione delle risorse (memoria, tempo processore, input/output ecc). Un processo non ha accesso allo spazio di indirizzamento degli altri, ma abbiamo l'illusione del parallelismo.

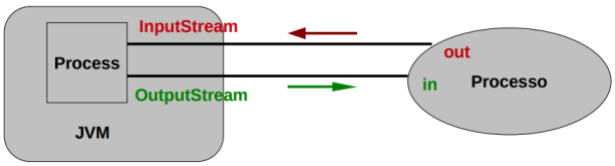
Java permette di manipolare dei processi, che non sono gestiti dalla JVM (Java Virtual Machine) ma dal sistema. Non c'è quindi una nozione di processo all'interno della JVM: un processo è un oggetto del sistema. Tuttavia, in un programma Java, possiamo dire al sistema di eseguire un processo e recuperare informazioni su di esso (come, ad esempio, i suoi input e output).
Per prima cosa, bisogna recuperare l'ambiente di esecuzione. È disponibile tramite un oggetto della classe **java.lang.Runtime**. Esiste solo un oggetto di questa classe, e non è possibile crearlo. Per recuperarlo, si usa la funzione **static Runtime.getRuntime().** Esistono molte funzioni per la classe java.lang.Runtime, ad esempio la funzione per avviare un processo, con la forma Process exec(...).
La JVM permette dunque di avviare processi esterni, che sono rappresentati sotto la forma di oggetti **java.lang.Process**. Questi oggetti servono a comunicare con i processi esterni che rappresentano e a sincronizzarsi con la loro esecuzione (per esempio per aspettare la fine di un processo).
Come si creano questi processi? Per esempio, per avviare un processo che esegue ls –a

```
String[] cmd={"ls","-a"};
Process process = Runtime.getRuntime().exec(cmd);
```

Comunicare con i processi

Vogliamo essere in grado di scambiare informazioni con i processi, per esempio per leggere cosa stampano e farli stampare dalla JVM. Per questo ci sono dei metodi che usano dei flussi:

- **InputStream getInputStream( )**
- **OutputStream getOutputStream()**



Gli input/output usano dei flussi (streams): gli Input stream permettono di leggere dei dati e gli Output stream permettono di scrivere dei dati. Esistono diverse classi di stream in base ai dati manipolati, per esempio:

- **java.io.FileInputStream**
- **sun.net.TelnetOutputStream**

Tutti gli Input stream usano delle funzioni simile di lettura e tutti gli Output stream usano delle funzioni simile di scrittura. Si usano dei filtri per facilitare la manipolazione dei dati, ad esempio per manipolare delle String piuttosto che dei byte[].

Output Stream

La classe di base per gli Output Stream è: **java.io.OutputStream** (abstract class)
Dispone delle funzioni:

- **public abstract void write(int b) throws IOException** //aspetta come input un intero b fra 0 e 255 e lo scrive sul flusso
- **public void write(byte[] data) throws IOException**
- **public void write(byte[] data, int offset, int length) throws IOException**
- **public void flush() throws IOException** //per svuotare il buffer associato allo stream
- **public void close() throws IOException**

La classe **java.io.OutputStreamWriter** estende la abstract class Writer. Si da al costruttore un **OutputStream (OutputStreamWriter(OutputStream out))**.
Esempi di funzioni di questa classe:

- **public void close() throws IOException**
- **public void flush() throws IOException**
- **public void write(int c) throws IOException**
- **public void write(char[] cbuf, int off, int len) throws IOException**
- **public void write(String str, int off, int len) throws IOException**

la abstract class **java.io.PrintWriter** è più piacevole da usare. Si da costruttore: **PrintWriter(Writer out).**
Esempi di funzioni di questa classe:

- **public void close() throws IOException** //chiude l'Output Stream corrispondente
- **public void flush() throws IOException**
- **public void print(char c), public void print(int i), public void print(String s), public void println(String s), public void print(Object o), ...**



Per riassumere, OutputStream viene usato per scrivere dei bytes, OutputStreamWriter per scrivere dei caratteri, e PrintWriter per scrivere ogni tipo di dato.

```
PrintWriter pw=new PrintWriter(new OutputStreamWriter(out));
pw.println("Hello");
```

Input Stream

La classe di base per gli Intput Stream è: **java.io.InputStream (abstract class).**
Dispone delle funzioni:

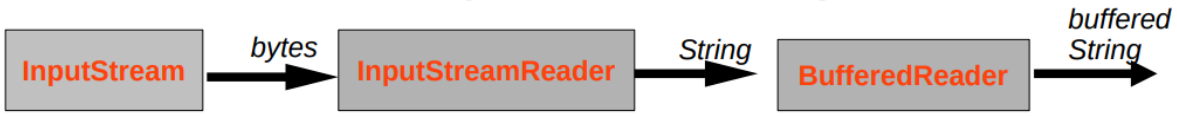
- **public abstract int read() throws IOException** //legge un intero e lo rimanda
- **public int read(byte[] input) throws IOException**
- **public int read(byte[] input, int offset, int length) throws IOException**
- **public void close() throws IOException**

La classe **java.io.InputStreamReader** estende la abstract classe Reader. Si dà al costruttore un **InputStream (InputStreamReader(InputStream in)).**
Esempi di funzione di questa classe:

- **public void close() throws IOException**
- **public int read() throws IOException**
- **public int read(char[] cbuf,int offset,int length) throws IOException**

La abstract class **java.io.BufferedReader** è più piacevole da usare. Si dà al costruttore: **PrintWriterBufferedReader(Reader in).**
Esempi di funzione di questa classe:

- **public void close() throws IOException** // chiude l'Input Stream corrispondente
- **public int read() throws IOException**
- **public int read(char[] cbuf,int offset,int length) throws IOException**
- **public String readLine()**



Per riassumere, InpupStream viene usato per leggere dei bytes, InputStreamReader per leggere dei caratteri, e BufferedReader per leggere dei caratteri tramite un buffer. I metodi di lettura aspettano dati da leggere se il flusso non è chiuso.

```
BufferedReader bf=new BufferedReader(new InputStreamReader(in));
Sring st=bf.readLine();
```

Come sincronizzarsi con un processo

Esistono due funzioni per sincronizzarsi con un processo (disponibili nella classe java.lang.Process)

- **int waitFor() throws InterruptedException**: aspetta la fine dell'esecuzione di un processo e restituisce il valore da lui restituito
- **int exitValue()**: restituisce il valore restituito dal processo

È importante aspettare la fine dell'esecuzione del processo, perché se il programma Java finisce prima, non possiamo più ricuperare le informazioni.
Come si passano le variabili a dei processi?

Per la programmazione concorrente non useremmo exec(...) della classe Runtime; come in C, piuttosto che usare processi del sistema useremmo dei threads che esistono nella JVM.
Un thread è un filo d'esecuzione all'interno di un programma. Il thread è eseguito da un processo, che può gestire più thread: si parla di processo multi-thread quando al minimo un thread è presente. Ogni filo d'esecuzione è diverso e ha come attributi un puntatore di esecuzione o PC (Program Counter) e uno stack d'esecuzione (stack). Un thread condivide tutto l'ambiente di esecuzione con gli altri threads. La JVM è multi-threaded e offre al programmatore la possibilità di manipolare i threads.
Per manipolare i threads in java, useremmo due classi importanti:

- L'interfaccia **java.lang.Runnable**: contiene una sola funzione da implementare, void run(), funzione che contiene il codice da eseguire dal thread
- La classe **java.lang.Thread**: permette di manipolare i thread

In sintesi, nella funzione run() si trova il codice del thread (la parte statica) e useremmo un oggetto Thread per gestire il filo d'esecuzione (la parte dinamica).
Ad ogni thread viene associato un oggetto che implementa l'interfaccia Runnable. Ad esempio, uno dei costruttori in java.lang.Thread è public **Thread(Runnable target)**. Lo stesso oggetto implementando Runnable può essere associato a più di un thread: in questo ultimo caso, ogni thread esegue in un modo concorrente la funzione run() dell'oggetto associato.
java.lang.Runnable è quindi un'interfaccia con una singola funzione da implementare: **public void run().** Quando un thread inizia la sua esecuzione, chiama la funzione run() del Runnable associato. Finisce la sua esecuzione quando questa funzione run() termina. Per avviare un thread non si chiama direttamente la funzione run() ma un'altra funzione chiamata start().

I threads Java hanno diversi attributi:

- String name: il nome del thread
- long id: l'identità del thread
- int priority: la sua priorità (i threads sono ordinati secondo questa priorità)
- boolean daemon: il suo modo di esecuzione (daemon o no, cf più lontano)
- Thread.state state: il suo stato (NEW, RUNNABLE, BLOCKED, WAITING, TERMINATED, ...)
- la sua stack della quale si può solo vedere lo stato
- il suo gruppo di thread

In **java.lang.Thread**, abbiamo i getters per questi attributi.

<u>Stati di un thread</u>

- Start: thread appena creato; quando viene chiamato start() entra nello stato ready-to-run
- Ready-to-run: pronto per essere eseguito
- Running: il thread è effettivamente in esecuzione; quando run() termina entra nello stato dead. Lo scheduler fa passare da ready-to-run a running e viceversa
- Dead: il thread può essere rimosso dal sistema quando run() termina o per eccezione non gestita
- Blocked: mentre è in running può entrare in questo stato, ad esempio in attesa di un input

- Sleeping: quando viene chiamata la funzione sleep(...). Rientra nello stato ready-to-run quando il tempo associato allo sleep è passato
- Waiting

Terminazione della JVM

Spesso si dice che un programma termina quando si esce dal main. In realtà, il programma non termina, ma è il processo associato che termina: non basta uscire dal main, bisogna prima terminare la prima chiamata al main (se per esempio ci sono state chiamate ricorsive). Bisogna ad aspettare che tutti i thread che non sono daemon finiscano. Un thread daemon è un thread di servizio in background progettato per eseguire attività di supporto. Esiste almeno uno thread daemon, come il garbage collector, oppure il thread responsabile per l'interfaccia grafica.

Creazione e controllo dei thread:

- Costruttori per creare un thread:
  - **Thread(Runnable target)**
  - **Thread(Runnable target, String name)**
- Funzioni per controllare l'esecuzione di un thread:
  - **void start()** // Permette di avviare il thread: chiamerà la funzione run() del Runnable associato
  - **void join()** // Permette di aspettare la fine dell'esecuzione del thread
  - **void interrupt()** // metto lo stato del thread in 'interrotto'; non ha nessuno effetto diretto (permette solo al thread di sapere che un altro thread desidera interromperlo)

IMPORTANTE: Non ci sono funzioni per fermare un thread, bisogna a fare in modo che finisca la sua prima chiamata a run.

- Funzioni static di Thread:
  - **Thread currentThread()** // Per recuperare l'oggetto thread che esegue la riga; utile per esempio per recuperare il nome
  - **boolean isInterrupted()** // Per testare se il thread ha ricevuto una richiesta di interruzione
  - **void sleep(long ms)** // Per far dormire il thread per un numero di millisecondi (si tratta di un tempo minimo ma il thread potrebbe dormire di più)
  - **void yield()** // Il thread rinuncia temporaneamente alla sua esecuzione e il suo stato diventa ready-to-run

Programmazione concorrente nei thread

La parola chiave **synchronized** viene usata nel codice per proteggere l'accesso a un parte del codice e garantire che al massimo un thread esegua il codice. Corrisponde a un lock (ogni oggetto Java ha il suo lock). Ci sono due modi per usarla:

- Si dichiara una funzione synchronized: **public synchronized int f(int a){...}**. Ogni chiamata synchronized dello stesso oggetto non può essere eseguita in parallelo
- Oppure si controlla un pezzo di codice **synchronized(object) {.../*codice da proteggere*/...}** e si indica l'oggetto per il quale si vuole usare il lock.

```
import java.io.*;
import java.lang.*;

public class CodeCounter implements Runnable{

    private Counter c;

    public CodeCounter(Counter _c){
        this.c=_c;
    }

    public void run(){
        for(int i=0; i<10000; i++){
         synchronized(c){
          c.setVal(c.getVal()+1);
         }
        }
    }
}
```

È importante pensare bene a dove mettere gli synchronized e ricordarsi che i lock sono associati a degli oggetti. Se due pezzi di codice usano la funzione synchronized dello stesso oggetto non saranno eseguiti in parallelo. Si deve mettere in synchronized solo le parti di codice che finiscono (altrimenti il lock rimane preso per sempre).
synchronized int f(...) {...} è la stessa cosa di **int f(...) { synchronized(this){...}}**: il lock usato è sempre il lock dell'oggetto associato.

Problema Producer/Consumer

Nel problema dei produttori/consumatori, i produttori scrivono in una variabile e i consumatori leggono i valori scritti. Ogni valore deve essere letto una unica volta. Se i consumatori sono più veloci, allora i valori possono essere letti più di una volta; se i produttori sono più veloci, alcuni valori possono essere persi. Ovviamente, produttori e consumatori non possono scrivere e leggere allo stesso tempo.

- Prima soluzione: Si crea un oggetto **SharedVariable** che contiene un valore val e un boolean **readyToWrite.** Se il boolean è true, si può scrivere e poi si imposta a false. Se il boolean è false, si può leggere la variabile e poi si imposta true. Però c'è un problema: c'è un'attesa attiva con i cicli while, e un producer o consumer potrebbe rimanere per sempre nel ciclo senza rilasciare il lock della variabile.
- Soluzione: Usare i metodi wait(), notify() et notifyAll(): wait() permette di rilasciare il lock e aspettare una notifica; notify()/notifyAll() permette di svegliare UNO/TUTTI i thread in attesa che possa/possono riprovare a prendere il lock.

```
public synchronized int read(){
        try{
        while(readyToWrite==true){wait();}
        readyToWrite=true;
        notifyAll();
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
        return val;
    }

    public synchronized void write(int v){
        try{
        while(readyToWrite==false){wait();}
        readyToWrite=false;
        notifyAll();
        val=v;
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
```

Programmazione Rete in Java

Ci sono due modi principali per comunicare:

- Comunicazione per flussi (streaming), come TCP: le informazioni sono trasmesse in ordine, non c'è perdita dei messaggi, ma ci sono degli inconvenienti: necessita una connessione e di risorse per gestire la correttezza della comunicazione. Il modello è di tipo Client-Server: un server attende una connessione, poi comunica con il processo che ha richiesto la connessione. Avere un server multi-threaded permette di gestire più connessioni in parallelo.
- Comunicazione per pacchetti (datagram), come UDP: l'ordine nella trasmissione dei pacchetti non è garantito; un pacchetto mandato per primo può arrivare dopo un pacchetto mandato in un secondo tempo, e non ci sono garanzie, ossia un pacchetto mandato può essere perso. Un processo attende messaggi su un port. I messaggi possono arrivare da diversi processi, e secondo i messaggi ricevuti fa delle azioni.